PIMPRI CHINCHWAD COLLEGE OF ENGINEERING (PCCOE), PUNE

Department of Information Technology | S.Y. B.Tech

Comprehensive Code Walkthrough & Viva Study Guide

Complete Code Walkthrough & OOP Notes

A Detailed, Class-by-Class Architecture & Method Guide for the Exam Timetable & Seating Generator

---

This document serves as an exhaustive reference explaining every package, class, field, constructor, algorithm, and method in the project, specifically designed for students facing internal assessments and external viva examinations.

Subject: Object Oriented Programming (Java)

Author: Second Year IT Student

JDK Version: Java SE 17+

Repository Link: amritakharasne62006-design/oops_project-

Coverage: 100% of Project Classes

1. Architecture Overview & Package Directory

The application is designed using a clean layered architecture under package `com.examscheduler`:

Package	Contained Classes	Primary Responsibility
`model`	`Person`, `Student`, `Faculty`, `Course`, `Room`, `TimeSlot`, `Exam`, `SeatAllocation`, `InvigilatorDuty`	Encapsulated data entities holding domain states, constructors (default, parameterized, copy), and getters/setters.
`strategy`	`SeatingStrategy`, `MixedSubjectSeating`, `ZigZagSeating`, `SequentialSeating`, `RandomSeating`	Implementation of the Strategy Pattern providing polymorphic seating arrangements.
`service`	`TimetableGenerator`, `SeatingService`, `InvigilatorService`	Core algorithmic processing (Graph Colouring, room capacity allocation, priority queue fairness).
`validation`	`ConflictChecker`	Auditing engine checking hard clashes (dual exams, room overflow, overlaps) and soft warnings.
`exception`	`SchedulingClashException`, `RoomCapacityExceededException`, `InsufficientInvigilatorsException`, `InvalidDateException`	User-defined checked exceptions for robust error handling.
`report`	`ReportGenerator`	Tabular console rendering of timetable, hall ticket, seating chart, duty roster, and stats.
`util`	`FileHandler`	Character stream file reading and CSV export using `BufferedReader` and `BufferedWriter`.
`default`	`Main`	Interactive 12-option Scanner menu driving the entire application workflow.

2. Model Package: Deep Class Breakdown

Class 1: Person.java (Abstract Base Class)

Abstraction | Encapsulation | Constructor Overloading | this Keyword

What it does: Models common attributes for any human in the examination ecosystem.

Attributes: `protected String id;`, `protected String name;`, `protected int age;` (protected so `Student` and `Faculty` can access them directly).

Key Methods:

- `Person()`: Default constructor. Uses `this("UNKNOWN", "Not Assigned", 18);` for constructor chaining.
- `Person(String id, String name, int age)`: Parameterized constructor. Uses `this.name = name;` to resolve name shadowing.
- `Person(Person other)`: Copy constructor. Deep copies fields from another Person instance.
- `public abstract void displayDetails();`: Abstract method enforcing custom profile displays in child classes.

Class 2: Student.java (Extends Person)

Inheritance | super Keyword | Method Overriding

What it does: Represents an examinee student, tracking PRN, division, semester, department, and enrolled courses.

Key Methods:

- `Student(prn, name, age, div, sem, dept)`: Invokes `super(prn, name, age)` to initialize base class fields, then sets student fields.
- `enrollCourse(Course c)`: Adds a course to `registeredCourses` list if not already present.
- `isEnrolledIn(String code)`: Returns `true` if student is registered for the specified course code. Used by Graph Colouring.
- `@Override public void displayDetails()`: Overrides Person's abstract method to format student profile and enrolled courses.

Class 3: Faculty.java (Extends Person, Implements Comparable)

Inheritance | Interfaces (Comparable) | Method Overriding

What it does: Represents an invigilator teacher. Keeps track of `dutyCount` and approved `leaveDates`.

Key Methods:

- `@Override public int compareTo(Faculty other)`: Compares `this.dutyCount` with `other.dutyCount`. Enables min-heap ordering in `PriorityQueue`!
- `addLeaveDate(String date)` & `isOnLeave(String date)`: Manages and validates faculty leave records.
- `incrementDuty()`: Increments duty count by 1 upon assignment.

Class 4: Course.java

Encapsulation | equals & hashCode

What it does: Encapsulates an academic subject: code (e.g. "CS201"), name, duration in minutes, and exam type ("Theory"/"Practical").

Key Methods: Overrides `equals(Object o)` and `hashCode()` based on `code`. This is required so courses can serve as unique keys in HashMaps for the conflict graph.

Class 5: Room.java

Encapsulation | 2D Grid Representation

What it does: Represents an exam hall with room number, capacity, rows, columns, and availability status.

Key Methods: `getGridCapacity()` returns theoretical capacity ($rows imes cols$).

Class 6: TimeSlot.java

Encapsulation

What it does: Represents an exam slot with slotId, date (YYYY-MM-DD), startTime (e.g., "10:00 AM"), endTime, and session label.

Classes 7-9: Exam, SeatAllocation, InvigilatorDuty

Composition / Aggregation

Exam.java: Connects a Course, a TimeSlot, a list of assigned Rooms, and a list of eligible Students.

SeatAllocation.java: Connects a Student, Course, Room, Row, Column, and a human-readable seat code (e.g., "R1-C2").

InvigilatorDuty.java: Connects an assigned Faculty teacher, Room, Exam, and TimeSlot.

3. Strategy Pattern Package: Seating Arrangements

Interface: SeatingStrategy.java

Interfaces | Abstraction | Polymorphism

```
public interface SeatingStrategy {
    List<SeatAllocation> allocate(List<Student> students, Map<Student, Course> courseMap, Room room);
    String getStrategyName();
}
```

Four concrete implementations demonstrate the Strategy Pattern:

1. **MixedSubjectSeating.java (Anti-Malpractice, Recommended):** Partitions students into separate queues based on course code. Fills the room row-by-row, alternating between course queues at every desk. Direct physical neighbors (front, back, left, right) never take the same exam paper!
2. **ZigZagSeating.java (Alternating Snake):** Fills even rows from left-to-right ($c=0 o cols-1$) and odd rows from right-to-left ($c=cols-1 o 0$). Breaks vertical line copying patterns.
3. **SequentialSeating.java:** Standard row-by-row, left-to-right sequential seat assignment.
4. **RandomSeating.java:** Uses `Collections.shuffle()` with a fixed seed to randomize desk placement unpredictably.

4. Service Layer: Core Algorithmic Logic

Service 1: TimetableGenerator.java (Graph Colouring)

Graph Theory | Greedy Algorithm | Clash Prevention

Logic Flow:

1. `buildConflictGraph()` : Loops over all course pairs (C_i, C_j). If any student is enrolled in both, an undirected edge is added to `conflictGraph` .
2. `generateSchedule()` :
   - For each course, inspects neighbor courses in the graph.
   - Gathers colors already assigned to neighbors.
   - Assigns the smallest available integer color $c \in \{0, 1, 2, \dots\}$.
   - Maps color c to `TimeSlot[c]` . Throws `SchedulingClashException` if colors exceed available slots.
3. `manualOverride(exam, newSlot, allExams)` : Re-validates whether moving an exam causes an edge conflict with other exams in that slot. Rejects with an exception if a clash occurs!

Service 2: SeatingService.java

Strategy Pattern | 2D Arrays

Logic Flow:

1. Sorts rooms by capacity descending (fills large auditoriums first for efficient hall management).
2. Checks total student count against total room capacity; throws `RoomCapacityExceededException` if exceeded.
3. Slices batches of students per room and invokes `currentStrategy.allocate(roomBatch, courseMap, room)` (Runtime Polymorphism).
4. `printRoomGrid(room, allocations)` : Allocates a `String[rows][cols]` matrix, fills it with student seat labels, and prints a formatted visual ASCII desk grid.

Service 3: InvigilatorService.java (PriorityQueue Fairness)

PriorityQueue (Min-Heap) Fair Workload

Logic Flow:

1. Initializes `PriorityQueue<Faculty>` ordered by `dutyCount` ascending.
2. For each exam room in each slot:
   - Polls the faculty member with lowest duties from the queue.
   - Verifies constraints: Not already busy in this slot, not on leave, not teaching their own subject.
   - Creates `InvigilatorDuty`, increments `dutyCount++`, and re-inserts into PriorityQueue.
3. Guarantees even distribution of work across all professors.

5. Validation & File I/O Layers

ConflictChecker.java (Constraint Engine)

Audit Engine | Hard vs. Soft Constraints

Audits the entire schedule and returns a list of issues:

- **Hard Constraints:** Student having 2 exams at the same time, exam on a Sunday/holiday, faculty double-booked in two rooms.
- **Soft Constraints:** Warnings if a student has back-to-back exams on the same date.

FileHandler.java (Character Streams)

Character Streams | FileReader & BufferedReader | FileWriter & BufferedWriter | try-catch-finally

Handles saving and loading data:

- `exportTimetableToCSV(exams, path)` : Uses `BufferedWriter` to write comma-separated rows. Resource closing is guaranteed inside a `finally` block.
- `exportSeatingToCSV(allocations, path)` : Uses `try-with-resources` to export seating allocations.
- `readTextFile(path)` : Uses `BufferedReader.readLine()` to read text files line-by-line.

6. Summary Viva Quick-Reference Cheat Sheet

Concept	Definition	Where Used in This Project
Default Constructor	Constructor taking no parameters; sets default values.	`Person()` , `Student()` , `Course()`
Parameterized Constructor	Constructor taking arguments to initialize specific values.	`Student(prn, name, age, div, sem, dept)`
Copy Constructor	Constructor taking an object of the same class to clone state.	`Person(Person other)` , `Student(Student other)`
`this` vs `super`	`this` references current instance; `super` references parent class.	`this.name = name;` vs `super(prn, name, age);`
Abstract Class vs Interface	Abstract class can have state/constructors; Interface defines pure contracts.	`Person` (abstract class) vs `SeatingStrategy` (interface)
Method Overloading vs Overriding	Overloading: same name, different parameters (compile-time). Overriding: same name & parameters in child class (runtime).	Constructor overloading in `Person` vs Overriding `displayDetails()` in `Student`
Checked vs Unchecked Exception	Checked: extends `Exception` , verified at compile-time. Unchecked: extends `RuntimeException` .	`SchedulingClashException extends Exception` (checked)
Character vs Byte Streams	Character streams handle 16-bit Unicode (text/CSV); Byte streams handle 8-bit raw bytes.	`BufferedReader` / `BufferedWriter` in `FileHandler`